

# DAI SIMBOLI ALLE PAROLE

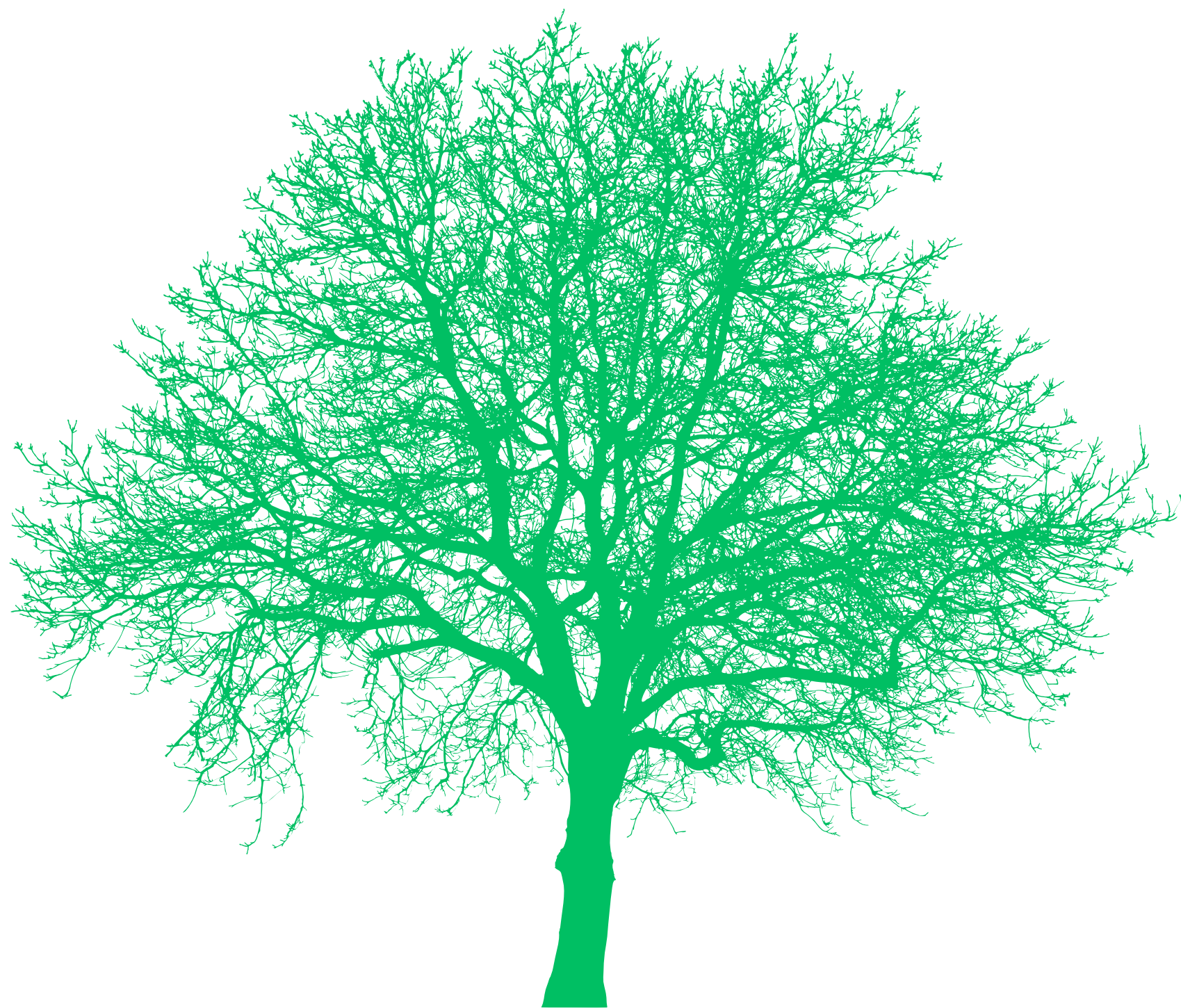
TRADUTTORE DI COMMENTI MUSICALI

---

DA EMOJI, EMOTICON E TESTO A  
FRASI IN ITALIANO



Università  
degli Studi  
di Palermo



LINGUAGGI E TRADUTTORI  
**LUCA PAVONE**  
0822483

## + Introduzione

Sui social esprimiamo i nostri gusti musicali in modo rapido e informale, mescolando **parole**, **emoji** ed **emoticon**.

È un linguaggio **immediato** per chi scrive, ma **non strutturato** e ambiguo: gli stessi concetti si possono esprimere a parole, con simboli o in forma mista.

Questo progetto realizza un piccolo traduttore che, tramite un analizzatore lessicale (**Flex**) e uno sintattico (**Bison**), riconosce questi commenti e li riscrive in frasi italiane complete e grammaticalmente corrette.

## + Obiettivo

Tradurre un commento musicale, in forma simbolica, testuale o ibrida, in una frase italiana **ben formata**: esplicitando il *soggetto sottinteso*, *convertendo* emoji ed emoticon in *parole* e gestendo *intensità*, *congiunzioni*, *domande* ed **errori**.

### Esempio

-   ? → secondo te questa canzone è **bomba** perché è **elettronica** ?
-   → questa canzone è **bomba** perché è **elettronica**
- >:( → questa canzone è **tremenda**
- :)( → **Errore lessicale: token non riconosciuto.**
-    → questa canzone è **molto ritmata** perché è **pop**

## + Architettura

Il sistema segue la classica pipeline in due fasi:

1. un analizzatore lessicale (Flex): legge l'input **carattere** per **carattere** e lo raggruppa in **token**, associando a ciascuno un **valore** ;
2. un analizzatore sintattico (Bison): riceve questi **token** e verifica che formino una **frase valida** secondo la **grammatica**, eseguendo le azioni che ne producono la traduzione.

Le due fasi lavorano in cascata e sono guidate dal **parser**: è quest'ultimo a chiedere un token alla volta al lexer tramite **yylex()**. Il risultato finale viene stampato a schermo e scritto nel file *output.txt*.

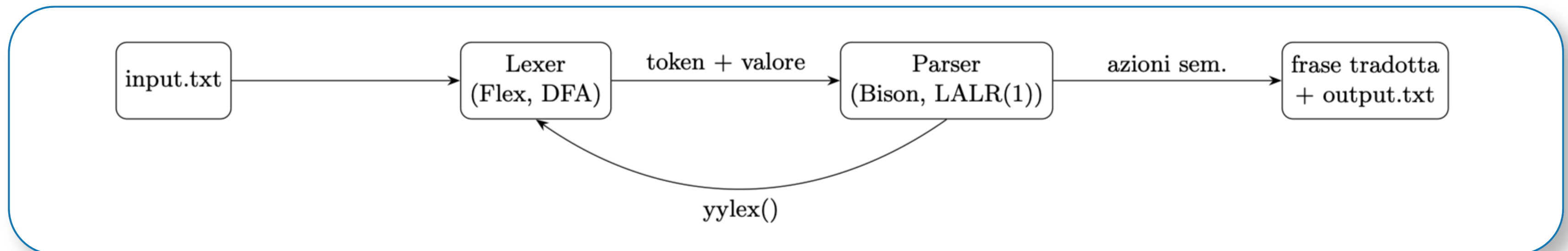
## + Tipo di Parser

Bison genera un **parser LALR(1): bottom-up**, di tipo **shift-reduce**, con **un** solo **token** di **lookahead**.

Il **lexer**, invece, è basato su un **automa a stati finiti deterministico** con la regola del longest match. La traduzione è guidata dalla sintassi:

*a ogni regola è associata un'azione semantica che calcola un attributo sintetizzato (una stringa).*

Così la frase tradotta si compone durante le riduzioni, risalendo dai figli al padre fino al simbolo iniziale che contiene il risultato.





```

F → PRE CORPO
    | CORPO
    | CORPO token_dom
    | PRE CORPO token_dom
CORPO → DESC GEN | CAU DESC
PRE → token_v_op
    | token_pro_rif token_v_im
    | token_pro_per token_v_op
    | token_pro_per
DESC → EL | EL token_cong DESC
EL → DB
    | token_avv_q EL
    | DB RIP
RIP → DB | DB RIP
DB → token_agg | token_emoji | EM
EM → token_emoticon | token_emoticon CODA
CODA → token_coda | token_coda CODA
GEN → CAU | ε
CAU → SEQ_G | token_motiv SEQ_G
SEQ_G → GB | GB RIP_G
RIP_G → GB | GB RIP_G
GB → token_genere
  
```

## + La Grammatica

La grammatica lavora quindi sui token forniti dal lexer, non sui caratteri grezzi.

Una frase è definita come:

- **prefisso opzionale:** chi commenta, ad esempio "io penso" o "mi sembra"
- **corpo:** cioè la **descrizione** della canzone con l'eventuale genere musicale, che possono comparire in **ordine libero** (bomba house o house bomba).

All'interno della descrizione, ogni **descrittore** (parola, emoji o emoticon) può essere preceduto da uno o più **avverbi di quantità**, *ripetuto* per esprimere intensità (più emoji uguali diventano "molto molto...") oppure unito a un altro descrittore tramite la congiunzione "e".

Le produzioni qui accanto formalizzano questa struttura; la legenda completa dei simboli è nella pagina seguente.

| Simbolo    | Descrizione                                 |
|------------|---------------------------------------------|
| $F$        | Simbolo iniziale (frase)                    |
| PRE        | Prefisso (chi commenta)                     |
| CORPO      | Corpo: descrizione + genere (ordine libero) |
| DESC       | Descrizione della canzone                   |
| EL         | Elemento descrittivo                        |
| RIP        | Ripetizione di descrittori uguali           |
| DB         | Descrittore base                            |
| EM         | Emoticon completa                           |
| CODA       | Coda dell'emoticon                          |
| GEN        | Genere musicale (opzionale)                 |
| CAU        | Causa (genere, con/senza "perché")          |
| SEQ_G      | Sequenza di generi                          |
| RIP_G      | Ripetizione di generi                       |
| GB         | Genere base                                 |
| $\epsilon$ | Stringa vuota (Epsilon)                     |
| token_*    | Elemento terminale (token)                  |

+ I Simboli

La tabella riporta la legenda dei simboli **non terminali** e **terminali** usati nelle produzioni della pagina precedente.

I **non terminali**, rappresentano le componenti astratte della frase: *il simbolo iniziale, il prefisso, il corpo, la descrizione, il genere* e così via, e si espandono in altri simboli secondo le regole della grammatica.

I terminali, indicati con la forma **token\_\***, sono invece gli elementi prodotti dall'analizzatore lessicale a partire dall'input (parole, emoji, emoticon, punteggiatura): costituiscono **le foglie dell'albero di derivazione** e non si espandono ulteriormente.

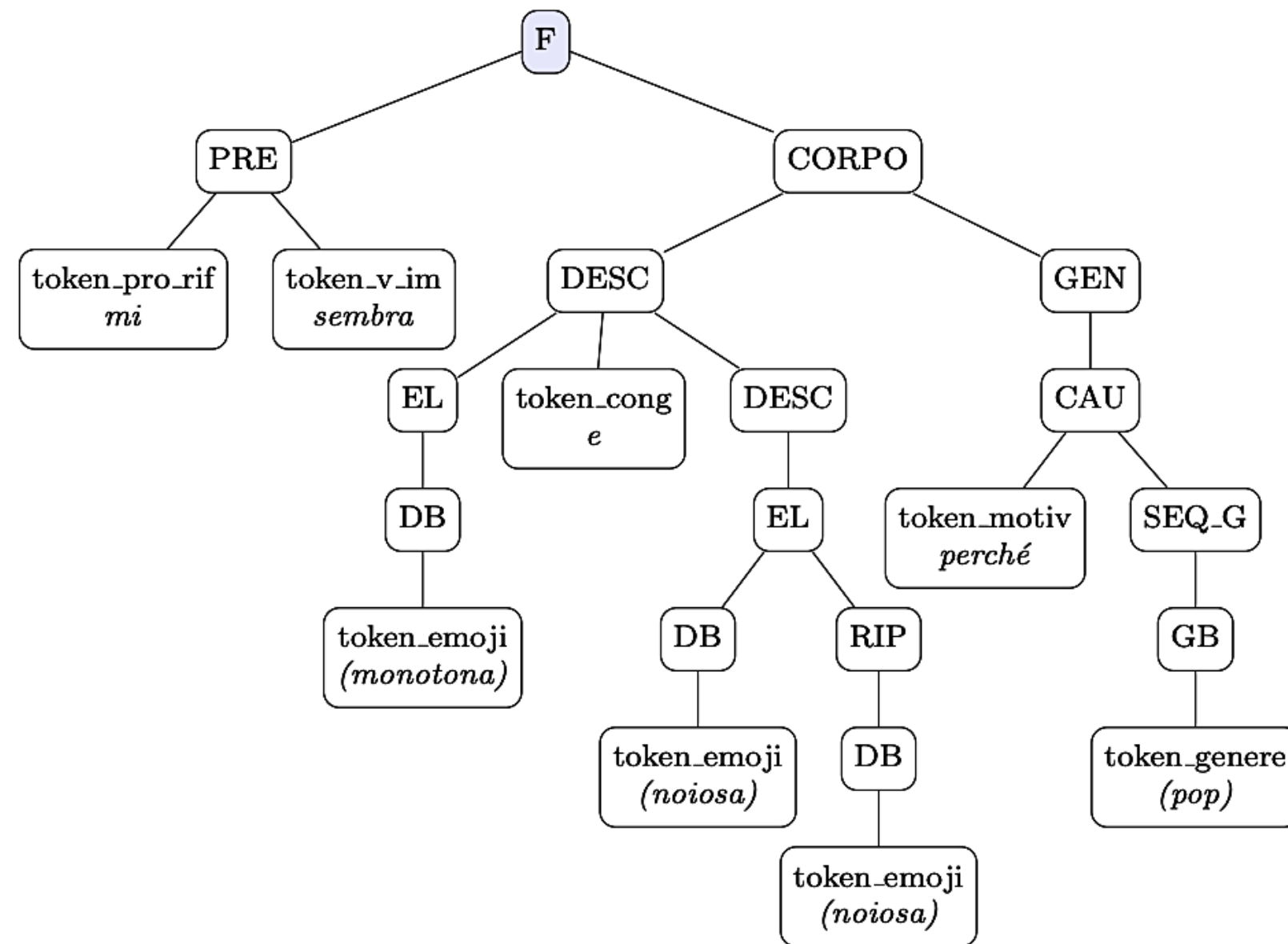
Il simbolo  $\epsilon$ , infine, permette la stringa vuota e segnala le parti opzionali della grammatica, cioè quelle che possono anche essere del tutto assenti come il genere musicale o il prefisso.

Input

Lexer Output

```
input.txt x
src > input.txt
1 🌪️
2 🌪️🌪️🌪️
3 molto 🌪️
4 bella
5 🌪️ e 🔥
6 io penso 🌪️
7 mi sembra 🔥
8 🌪️ e 🌪️
9 🌪️
10 🌪️ 🌪️
11 🌪️ 🌪️ 🌪️
12 🌪️ 🌪️ ?
13 io penso bella perche techno
14 troppo pazzesca 🌪️
15 :)
16 :)))
17 :(
18 >:(
19 mi sembra 🌪️ e 🌪️🌪️ perché 🌪️
20 🌪️ 🌪️
21 >:( e 🌪️🌪️ perché 🌪️
22 io pensi bomba
23 🌪️ ? 🌪️
24 :)(
25 hey
26
27
28
```

```
[EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [EMOJI: 🌪️] [EMOJI: 🌪️] [NEWLINE]
[PAROLA: molto] [EMOJI: 🌪️] [NEWLINE]
[PAROLA: bella] [NEWLINE]
[EMOJI: 🌪️] [PAROLA: e] [EMOJI: 🔥] [NEWLINE]
[PAROLA: io] [PAROLA: penso] [EMOJI: 🌪️] [NEWLINE]
[PAROLA: mi] [PAROLA: sembra] [EMOJI: 🔥] [NEWLINE]
[EMOJI: 🌪️] [PAROLA: e] [EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [EMOJI: 🌪️] [EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [EMOJI: 🌪️] [PUNTEGGIATURA: ?] [NEWLINE]
[PAROLA: io] [PAROLA: penso] [PAROLA: bella] [PAROLA: perche] [PAROLA: techno] [NEWLINE]
[PAROLA: troppo] [PAROLA: pazzesca] [EMOJI: 🌪️] [NEWLINE]
[EMOTICON: :)] [NEWLINE]
[EMOTICON: :)] [SIMBOLO: )] [SIMBOLO: )] [SIMBOLO: )] [NEWLINE]
[EMOTICON: :(] [NEWLINE]
[EMOTICON: >:(] [NEWLINE]
[PAROLA: mi] [PAROLA: sembra] [EMOJI: 🌪️] [PAROLA: e] [EMOJI: 🌪️] [EMOJI: 🌪️] [PAROLA: perché] [EMOJI: 🌪️] [NEWLINE]
[EMOJI: 🌪️] [EMOJI: 🌪️] [NEWLINE]
[EMOTICON: >:(] [PAROLA: e] [EMOJI: 🌪️] [EMOJI: 🌪️] [PAROLA: perché] [EMOJI: 🌪️] [NEWLINE]
[PAROLA: io] [PAROLA: pensi] [PAROLA: bomba] [NEWLINE]
[EMOJI: 🌪️] [PUNTEGGIATURA: ?] [EMOJI: 🌪️] [NEWLINE]
[ERRORE: :)(] [NEWLINE]
Token non riconosciuto: h
[ERRORE: h] [PAROLA: e] Token non riconosciuto: y
[ERRORE: y] [NEWLINE]
```



## + I Simboli

Una volta verificata la struttura della frase, la traduzione viene costruita **bottom-up**: a ogni regola della grammatica è associata un'azione semantica che calcola una stringa (un attributo sintetizzato), e risalendo l'albero queste stringhe si combinano fino a formare la frase finale nel simbolo iniziale.

## + Scelte progettuali principali

- il soggetto sottinteso viene reso **esplicito** ("questa canzone è...").
- L'accordo tra **soggetto** e **verbo** è controllato con **semplici if** che, in caso di incoerenza, segnalano un errore (YYERROR);
- le **emoji ripetute** vengono interpretate come intensità ("molto molto...")
- descrittori diversi vengono uniti dalla congiunzione "e".

L'albero in figura riassume tutti questi casi: dal prefisso "mi sembra" nasce "secondo me questa canzone è...", le due emoji diverse sono collegate dalla "e", e la coppia ripetuta diventa "molto noiosa".

INPUT

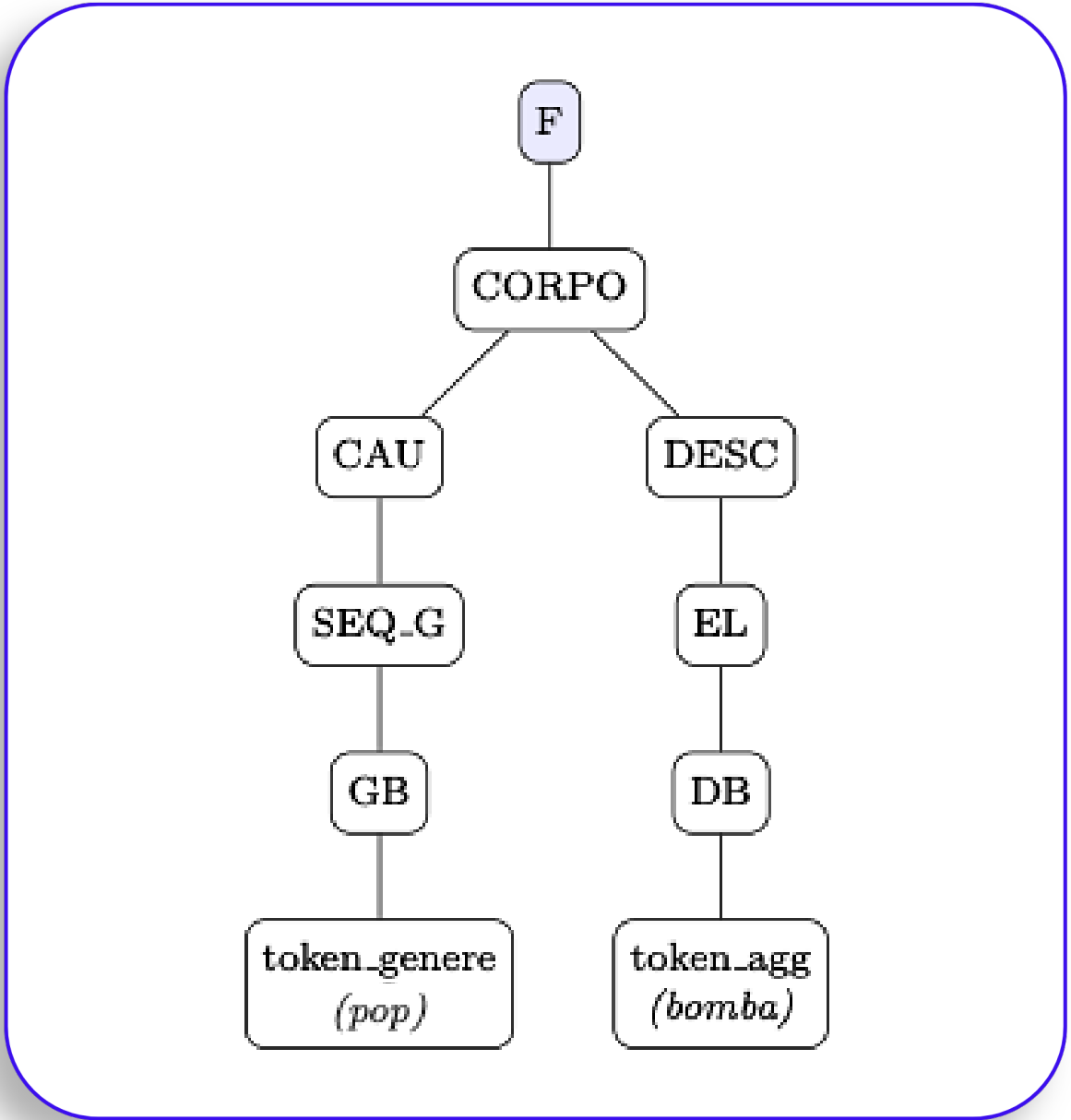
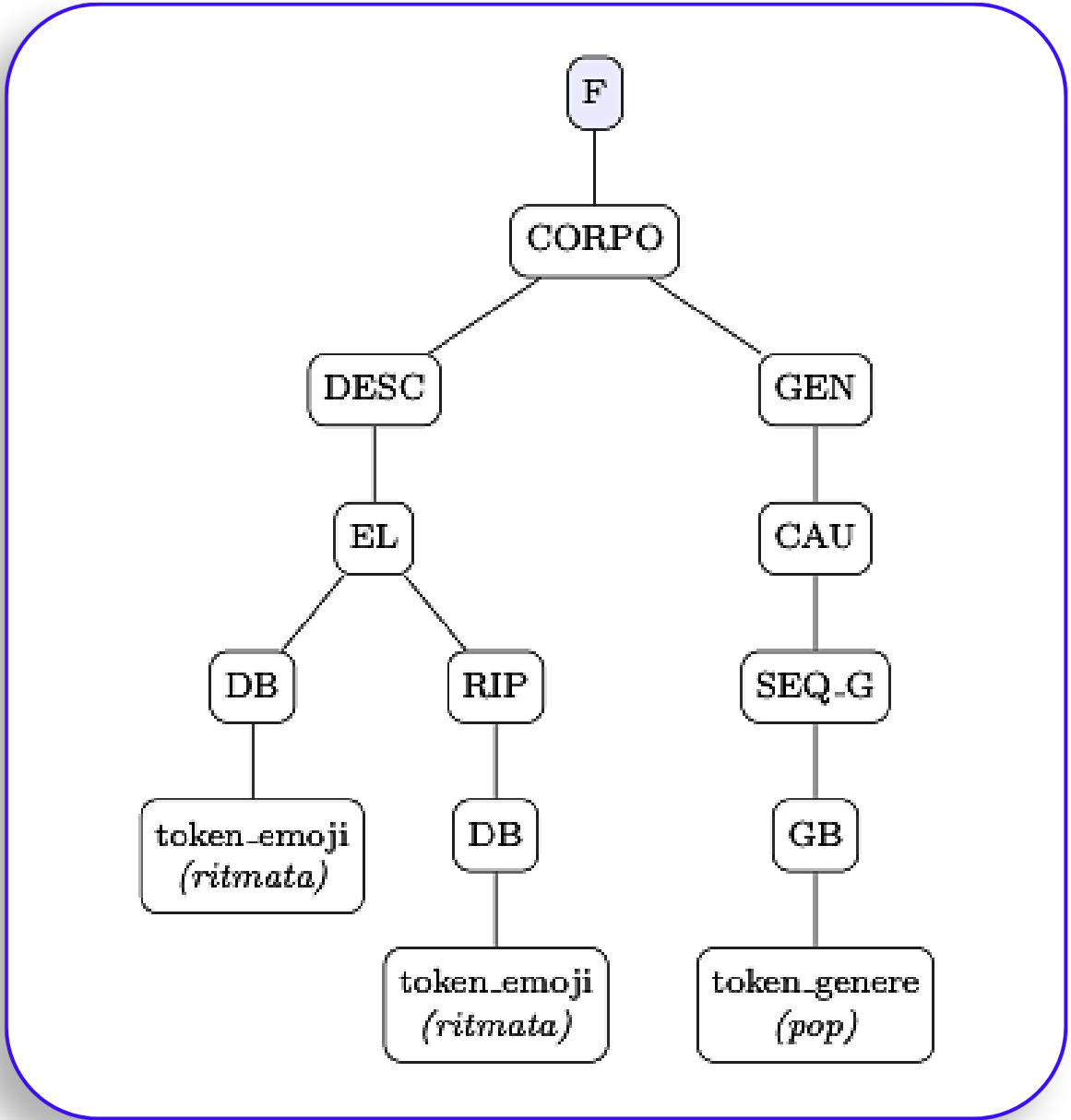
mi sembra 🙄 e 🤔🤔 perché 🎸

→

OUTPUT

"secondo me questa canzone è **monotona** e **molto noiosa** perché è **pop**"

+ Ulteriori Esempi



INPUT



"questa canzone è **molto ritmata** perché è **pop**"

OUTPUT

INPUT



"questa canzone è **bomba** perché è **pop**"

OUTPUT



## + Gestione degli errori

Il sistema riconosce due tipi di errore:

1. L'**errore lessicale** scatta quando l'analizzatore lessicale incontra qualcosa che non appartiene al suo dizionario chiuso: un simbolo o una parola sconosciuti (es. hey, o un'emoji non prevista) oppure un'emoticon malformata come “:)(“ e produce il token **ERRORE\_LESSICALE**.
2. L'**errore sintattico** scatta invece quando i token sono validi ma **la loro sequenza** non rispetta la **grammatica**: ad esempio un accordo soggetto/verbo sbagliato (io pensi bomba), due descrittori diversi attaccati senza la congiunzione "e" (🔥🔥), o un genere senza descrittore.

In entrambi i casi il programma stampa un messaggio **chiaro** e, grazie alla regola di recupero error **NEWLINE** con **yyerrok**, riprende dalla riga successiva: così una frase scorretta segnala il problema senza bloccare l'analisi del resto dell'input.

Input

```
input.txt x
src > input.txt
1  🌟
2  🌟🌟🌟
3  molto 🌟
4  bella
5  🌟 e 🔥
6  io penso 🌟
7  mi sembra 🔥
8  🌟 e 🌟
9  🌟
10 🌟 🌟
11 🌟 🌟 🌟
12 🌟 🌟 ?
13 io penso bella perche techno
14 troppo pazzesca 🌟
15 :)
16 :)))
17 :(
18 >:(
19 mi sembra 🌟 e 🌟 perché 🌟
20 🌟 🌟
21 >:( e 🌟 perché 🌟
22 io pensi bomba
23 🌟 ? 🔥
24 :)(
25 hey
26
27
28
```

Terminal Output

```
questa canzone è bomba
questa canzone è molto molto bomba
questa canzone è molto bomba
questa canzone è bella
questa canzone è bomba e infuocata
secondo me questa canzone è bomba
secondo me questa canzone è infuocata
questa canzone è bomba e ritmata
questa canzone è aliena
questa canzone è noiosa perché è house
questa canzone è molto ritmata perché è pop
secondo te questa canzone è bomba perché è elettronica?
secondo me questa canzone è bella perché è elettronica
questa canzone è troppo pazzesca perché è metal
questa canzone è bella
questa canzone è molto molto molto bella
questa canzone è brutta
questa canzone è tremenda
secondo me questa canzone è monotona e molto noiosa perché è pop
questa canzone è bomba perché è pop
questa canzone è tremenda e molto noiosa perché è pop
Errore sintattico: frase non conforme alla grammatica definita.
Errore sintattico: frase non conforme alla grammatica definita.
Errore lessicale: token non riconosciuto.
Token non riconosciuto: h
Token non riconosciuto: y
Errore sintattico: frase non conforme alla grammatica definita.
```